

EMBEDDED SYSTEMS (ES-371)

BATCH: 22C S

(6TH SEMESTER THIRD YEAR)

1

**CORTEX-M0+ARM PROCESSOR
INSTRUCTION SETS & ARM ASSEMBLY BASICS**

TEXT BOOKS

Embedded Systems Fundamentals with Arm Cortex-M based Microcontrollers: A Practical Approach (Latest Edition)

By: Dr Alexander G. Dean, Atmel Arm

The Definitive Guide to ARM® Cortex®-M0 and Cortex-M0+ Processors (Second Edition)

By: Joseph Yiu



OUTLINE

ARM and Thumb instruction sets

Cortex-M0+Instruction Set

ARM Assembly Basics

Pre-UAL and UAL

Instruction Sets Groups

Moving data within the Processor

Memory Access & Stack Memory Access

Arithmetic & Logical Operations

Shift and rotate Operations

Branch Instructions

Reverse and Extending Operations

Directives

ARM program code template



Instruction Sets

All processors carry out their require operations by executing sequences of instructions.

In simple processor designs, the following types of instructions are required:

Data processing (arithmetic operations like “add”/“subtract,” logic operations like “AND”/“OR”)

Memory access instructions (read memory, write memory)

Program flow control instructions (branches, conditional branches, function calls)

The ARM Cortex-M0 and Cortex-M0+ processors also have instructions for

Exception and OS support

Accesses to special registers

Sleep operations

Memory barriers



ARM and Thumb Instruction Set

Early ARM instruction set

32-bit instruction set, called the ARM instructions

Powerful and good performance

Larger program memory compared to 8-bit and 16-bit processors

Larger power consumption

Thumb-1 instruction set

16-bit instruction set, first used in ARM7TDMI processor in 1995

Provides a subset of the ARM instructions, giving better code density compared to 32-bit RISC architecture

Code size is reduced by $\sim 30\%$, but performance is also reduced by $\sim 20\%$



ARM and Thumb Instruction Set

Thumb-2 instruction set

Consists of both 32-bit Thumb instructions and original 16-bit Thumb-1 instruction sets

Compared to 32-bit ARM instructions set, code size is reduced by $\sim 26\%$, while keeping a similar performance

Capable of handling all processing requirements in one operation state



ARM and Thumb Instruction Set

Mix of ARM and Thumb-1 Instruction sets

Benefit from both 32-bit ARM (high performance) and 16-bit Thumb-1 (high code density).

A multiplexer is used to switch between two states: ARM state (32-bit) and Thumb state (16-bit), which requires a switching overhead.

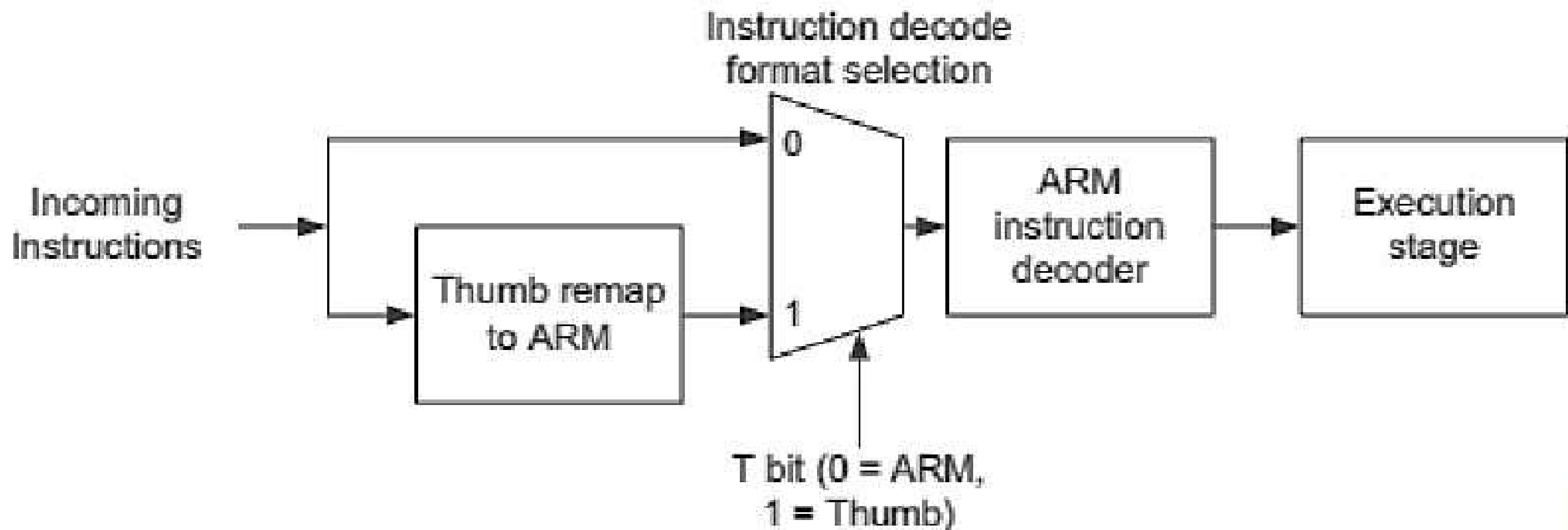


Figure 1: ARM7TDMI design supports both ARM and Thumb instruction set

ARM Assembly Syntax

For ARM assembly (applies to ARM Development Studio 5 and Keil Microcontroller Development Kit), the following instruction formatting is used:

Label

mnemonic *operand1,* *operand2, ...* *; Comments*

Label is used as a reference to an address location;

Mnemonic: is the name of the instruction;

Operand1 is the destination of the operation;

Operand2 is normally the source of the operation;

Comments are written after “ ; ”, which does not affect the program;

For example: Put an immediate constant value into a register using MOV instruction. Immediate data are usually prefixed with “#”:

MOV R0, #0x12 ; Set R0 = 0x12 (hexadecimal)

MOV R1, #A ; Set R1 = ASCII character A



ARM Assembly Syntax

Pre-UAL and UAL

Pre-UAL	UAL
Pre-Unified Assembler Language was previously used syntax for writing assembly code	Unified Assembler Language (UAL) is current used syntax for writing assembly code
Data operation instructions use two operands	Data operation instructions use three operands
The “S” suffix was not essential in the syntax of instructions that update the APSR Example: A Pre-UAL ADD instruction for 16-bit Thumb code is <i>ADD R0, R1</i> ; R0 = R0 + R1, ;update APSR	The syntax of instructions that update the APSR should have the “S” suffix Example: UAL syntax is <i>ADDs R0, R0, R1</i> ; R0 = R0 + R1, ;update APSR

Cortex-M0+ Instruction Set

ARMv6-M architecture profile

Includes all of the 16-bit Thumb instruction from ARMv7-M, excluding CBZ, CBNZ and IT as shown in Table 1.

Also supports 32-bit Thumb-2 instructions shown in Table 2.

Table 1: 16-bit Thumb instructions supported on the Cortex-M0 and
16-bit Thumb instructions supported on Cortex-M0/M0+ processors
Cortex-M0+ processor

ADC	ADD	ADR	AND	ASR	B	BIC	BLX	BKPT	BX
CMN	CMP	CPS	EOR	LDM	LDR	LDRH	LDRSH	LDRB	LDRSB
LSL	LSR	MOV	MVN	MUL	NOP	ORR	POP	PUSH	REV
REV16	REVSH	ROR	RSB	SBC	SEV	STM	STR	STRH	STRB
SUB	SVC	SXTB	SXTH	TST	UXTB	UXTH	WFE	WFI	YIELD

Table 2: 32-bit Thumb instructions supported on the Cortex-M0 and

32-bit Thumb instructions supported on Cortex-M0/M0+ processors

BL	DSB	DMB	ISB	MRS	MSR
----	-----	-----	-----	-----	-----

Cortex-M0+ Instruction Set Groups

The instructions in the Cortex-M0 and Cortex-M0+ Processors can be divided into various groups based on functionality:

- Moving data within the processor

- Memory Accesses

- Stack Memory Accesses

- Arithmetic operations

- Logic operations

- Shift and Rotate operations

- Extend and reverse ordering operations

- Program flow control (Branch, conditional branch, and function calls)

- Memory barrier instructions

- Exception-related instructions



MOVING DATA WITHIN THE PROCESSOR (MOV, MOV S AND MVNS)

In Thumb the instruction mnemonic for moving data is MOV.

Several types of MOV instructions, based on the operand type and opcode suffix.

Syntax:

MOV Rd, Rm ; Rd is the destination register, Rm is the source register

MOVS Rd, Rm ; Mov Rm into Rd

MOV S Rd, #imm ; imm is any value in the range 0-255

MVNS Rd, Rm ; inverse the value of Rm and put into Rd

Operation:

The MOV instruction copies the value of Rm into Rd.

The MOV S instruction performs the same operation as the MOV instruction, but also updates the N and Z flags.

The MOV S Rd, #imm Move immediate data (sign extended) into register

The MVNS instruction takes the value of Rm, performs a bitwise logical NOT operation on the value, and places the result into Rd

MOVING DATA WITHIN THE PROCESSOR (MOV, MOVS AND MVNS)

Restrictions – there is some restrictions using this instructions

Rm and Rd are both low registers (R0-R7).

When Rd is the Program Counter (PC) in a MOV instruction:

bit[0] of the result is ignored

A branch occurs to the address created by forcing bit[0] of that value to 0. The T-bit remains unmodified.

Note – it is possible to use MOV as a branch instruction, ARM strongly recommends the use of a BX or BLX instruction to branch for software portability to the ARM instruction set.

Condition flags – If S is specified, these instructions:

Update the N and Z flags according to the result

Do not affect the C or V flag



MOVING DATA WITHIN THE PROCESSOR (MOV, MOV S AND MVNS)

Examples:

MOV S R0, #0x000B ; Write value of 0x000B to R0, flags get updated

MOV S R7, #0x0 ; Write value of zero to R7, flags are updated

MOV R5, R7 ; Write value in R5 to R7, flags are not updated

MOV S R3, #23 ; Write value of 23 to R3

MVNS R2, R0 ; Write inverse of R0 to the R2 and update flags

MOV R7, #256 ; incorrect value because it support values from 0-255.

MOV R8, #25 ; incorrect because we cant use high register



MRS AND MSR INSTRUCTIONS

MRS: Move the contents of a special register to a general-purpose register.

MSR: Move the contents of a general-purpose register to special register.

Syntax:

MRS Rd, spec_reg

MSR spec_reg, Rn

Where:

- „Rd is the general-purpose destination register.
- „Rn is the general-purpose source register.
- „spec_reg is one of the special-purpose registers: APSR, IPSR, EPSR, IEPSR, IAPSR, EAPSR, PSR, MSP, PSP, PRIMASK, or CONTROL



MRS AND MSR INSTRUCTIONS

Operation:

MRS stores the contents of a special-purpose register to a general-purpose register. MRS can be combined with the MSR instruction to produce read-modify-write sequences, which are suitable for modifying a specific flag in the PSR.

MSR updates one of the special registers with the value from the register specified by Rn.

Restrictions:

Rd, Rn must not be SP or PC.

Condition flags

MRS instruction does not change the flags.

MSR instruction updates the flags explicitly based on the value in Rn.



SPECIAL REGISTER SYMBOLS FOR MRS AND MSR INSTRUCTIONS

Symbol	Register	Access type
APSR	Application Program Status Register (PSR)	Read/Write
EPSR	Execution PSR	No accesses (read as zero)
IPSR	Interrupt PSR	Read only
IAPSR	Composition of IPSR and APSR	Read only
EAPSR	Composition of EPSR and APSR	Read only (EPSR read as zero)
IEPSR	Composition of IPSR and EPSR	Read only (EPSR read as zero)
XPSR	Composition of APSR, EPSR, and IPSR	Read only (EPSR read as zero)
MSP	Main Stack Pointer	Read/Write
PSP	Process Stack Pointer	Read/Write
PRIMASK	Primary Exception Mask register	Read/Write
CONTROL	CONTROL register	Read/Write



MRS AND MSR INSTRUCTIONS EXAMPLES

MRS R0, PRIMASK ; Read PRIMASK value and write it to R0

MRS R9, PRIMASK ; Read PRIMASK register into R9

MRS R3, XPSR ; Read xPSR register into R3

MSR CONTROL, R1 ; Read R1 value and write it to the CONTROL register

MSR CONTROL, R0 ; Write R0 into CONTROL register

MSR PRIMASK, R9 ; Write R9 into PRIMASK register



MEMORY ACCESS INSTRUCTIONS

The Cortex-M0 and Cortex-M0+ processors support a number of memory access instructions, which support various data transfer sizes and addressing modes.

The supported data transfer sizes are Word, Half Word, and Byte. In addition, there are separate instructions to support signed and unsigned data.

Following Table summarizes the memory address instruction mnemonics for single load and store operations.

Transfer size	Unsigned load	Signed load	Signed/ Unsigned store
Word	LDR	LDR	STR
Half word	LDRH	LDRSH	STRH
Byte	LDRB	LDRSB	STRB



MEMORY ACCESS INSTRUCTIONS

The instructions listed in above Table, support multiple addressing modes.

When the instruction is used with different operands, different instruction encodings are generated by the assembler

Important Note:

It is important to make sure the memory address accessed is aligned.

For example, a word size access can only be carried out on address locations when address bits[1:0] are set to zero,

A half-word size access can only be carried out on address locations when address bit[0] is set to zero.

Unaligned transfers are not supported on the ARMv6-M Architecture (include Cortex-M0 and Cortex-M0+ processors). Any attempt at unaligned memory access result in a HardFault exception.

Byte size transfers are always aligned on the Cortex-M processors.



MEMORY ACCESS INSTRUCTIONS

MEMORY READ OPERATION

For memory read operations, the instruction to carry out single accesses is **LDR** (load):

Read single memory data into register:

Syntax:

LDR Rt, [Rn, Rm] ; signed/unsigned word read

LDRB Rt, [Rn, Rm] ; read byte

LDRH Rt, [Rn, Rm] ; read half word

Where:

Rt, Rn, Rm are low level registers

„Rt is the register to load or store $Rt = \text{memory}[Rn + Rm]$

„Rn is the register on which the memory address is based

„Rm is a register containing a value to be used as the offset



MEMORY ACCESS INSTRUCTIONS

MEMORY READ OPERATION

Read single memory data into register with immediate offset:

Syntax:

LDR Rt, [Rn, , #mmed5] ; word read

LDRH Rt, [Rn, #mmed5] ; half-word read

LDRB Rt, [Rn, #mmed5] ; Byte read

Where

$Rt = \text{memory}[Rn, + \text{ZeroExtend}(\#mmed5 \ll 2)]$; Word

$Rt = \text{memory}[Rn + \text{ZeroExtend}(\#mmed5 \ll 1)]$; Half word

$Rt = \text{memory}[Rn + \text{ZeroExtend}(\#mmed5)]$; Byte

Rt and Rn are low registers



MEMORY ACCESS INSTRUCTIONS

MEMORY READ OPERATION

Read single SIGNED memory data into register:

Syntax:

LDRSB Rt, [Rn, Rm] ;read signed byte

LDRSH Rt, [Rn, Rm] ;read signed half-word

Where:

Rt, Rn, Rm are low level registers

$Rt = \text{SignExtend}(\text{memory}[Rn+Rm])$



MEMORY ACCESS INSTRUCTIONS

MEMORY WRITE OPERATION

For memory write operation, the instruction is **STR** (store):

Write single register data into memory:

Syntax:

STR Rt, [Rn, Rm] ; signed/unsigned word write

STRB Rt, [Rn, Rm] ; byte write

STRH Rt, [Rn, Rm] ; half-word write

Where:

$\text{memory}[\text{Rn} + \text{Rm}] = \text{Rt}$

Rt, Rn, Rm are low level registers



MEMORY ACCESS INSTRUCTIONS

MEMORY WRITE OPERATION

Write single register data into memory with immediate offset:

Syntax:

STR Rt, [Rn, , #mmed5] ; word write

STRH Rt, [Rn, #mmed5] ; half-word write

STRB Rt, [Rn, #mmed5] ; Byte write

Where:

memory[Rn, + ZeroExtend (#mmed5 << 2)] = Rt ; Word

memory[Rn + ZeroExtend(#mmed5 << 1)] = Rt ; Half word

memory[Rn + ZeroExtend(#mmed5)] = Rt ; Byte

Rt and Rn are low registers



MEMORY ACCESS INSTRUCTIONS

LOAD MULTIPLE REGISTERS

For load/store multiple instructions, the transfer size is always in Word size.

Instruction: **LDM (Load Multiple)**

Function: Read multiple memory data word into registers, base address register update by memory read

Syntax :

LDM Rn, {Ra, Rb ,...} ; Load multiple registers from memory

Where:

$Ra \Rightarrow \text{memory}[Rn],$

$Rb \Rightarrow \text{memory}[Rn+4],$

$Rn, Ra, Rb \dots$, are low registers.

Rn is on the list of registers to be updated by memory read.

Example:

LDM R2, {R1, R2, R5 - R7} ; Read R1,R2,R5,R6, and R7 from memory



MEMORY ACCESS INSTRUCTIONS

LOAD MULTIPLE REGISTERS

Instruction: **LDMIA (Load Multiple Increment After)**

Function: Read multiple memory data word into registers and update base register

Syntax :

LDMIA Rn!, {Ra, Rb ,..} ; Load multiple registers from memory
;and increment base register after completion

Where:

Ra = memory[Rn],

Rb = memory[Rn+4],

.....

then update Rn to last read address plus 4.

Rn, Ra, Rb are low registers.

Example:

LDMIA R0!, {R1, R2, R5 - R7} ; Read multiple registers, R0 update
;to address last read operation



MEMORY ACCESS INSTRUCTIONS

STORE MULTIPLE REGISTERS

Instruction: **STMIA (Store Multiple Increment After)/STMEA**

Function: Write multiple register data into memory and update base register

Syntax:

STMIA Rn!, {Ra, Rb ,...} ; Store multiple registers to memory
 ; and increment base register after completion

Where:

memory[Rn] = Ra,
memory[Rn+4] =Rb,
.....

Then update Rn to last store address plus 4.

Rn, Ra, Rb ..., are low registers.

Example:

STMIA R0!, {R1, R2, R5 - R7} ; Store R1, R2, R5, R6, and R7 to memory
 ;and update R0 to address after where R7 stored

MEMORY ACCESS INSTRUCTIONS

STACK MEMORY ACCESS

There are two memory access instructions that are dedicated to stack memory accesses.

The **PUSH** instruction is used to decrement the current SP and store data to the stack.

The **POP** instruction is used to read the data from the stack and increment the current SP.

Both PUSH and POP instructions allow multiple registers to be stored or restored. However, only low registers, Link Register (LR) (for PUSH operation) and PC (for POP operation) are supported.



MEMORY ACCESS INSTRUCTIONS

STACK MEMORY ACCESS

Instruction: **PUSH**

Function: Write single or multiple registers (low register and Link Register (LR)) into memory and update base register (Stack Pointer (SP))

Syntax:

PUSH {Ra, Rb ,...} ; Store multiple registers to memory and
 ; decrement SP to the lowest pushed data address

PUSH {Ra, Rb, .., LR} ; Store multiple registers and LR to memory and
 ; decrement SP to the lowest pushed data address

Where:

$\text{new_SP} = \text{SP} - 4 \times \text{number of registers to PUSH}$
 $\text{memory}[\text{new_SP}] = \text{Ra},$
 $\text{memory}[\text{new_SP} + 4] = \text{Rb},$
.....
and then update SP to new_SP.

Example:

PUSH {R1, R2, R5 - R7, LR} ; Store R1, R2, R5, R6, R7, and LR
 ;to stack.

The order of the register content is based on registers number, i.e., Lower register is push to the lower address in the stack)

MEMORY ACCESS INSTRUCTIONS-STACK MEMORY ACCESS

Instruction: **POP**

Function: Read single or multiple registers (low register and Program Counter (PC)) from memory and update base register (Stack Pointer (SP))

Syntax:

POP {Ra, Rb ,...} ; Load multiple registers from memory and
 ;increment SP to the last emptied stack
 ;address plus 4

POP {Ra, Rb, .., PC} ; load multiple registers and PC from memory
 ;and increment SP to the last emptied stack
 ;address plus 4

Where:

Ra = memory[SP],

Rb =memory[SP+4]

.....

and then update SP to last restored address plus 4.

Example:

POP {R1, R2, R5 - R7} ; Restore R1, R2, R5, R6, R7 from stack



ARITHMETIC OPERATION

The Cortex-M0 and Cortex-M0+ Processors support a number of Arithmetic operations (add, subtract, twos complement, and multiply).

For most of these instructions, the operation can be carried out between two registers, or between one register and an immediate constant.

ADD Instruction

1. Add two registers, APSR update

ADDS Rd, Rn, Rm ; Rd = Rn + Rm, APSR update.

;Rd, Rn, Rm are low registers.

2. Add two registers without updating APSR

ADD Rd, Rn, Rm ; Rd = Rn + Rm,

;Rd, Rn, Rm must **low** registers.

3. Add an immediate constant into a register

ADDS Rd, Rn, #immed3

;Rd = Rn + ZeroExtend(#immed3),

;APSR update

ADDS Rd, #immed8

;Rd = Rd + ZeroExtend(#immed8), ;

;APSR update

;Rd, Rn, Rm are low registers.



ARITHMETIC OPERATION

4 Add with Carry and update APSR

ADCS Rd, Rm

; Rd=Rd+Rm+Carry

; Rd and Rm are low registers



ARITHMETIC OPERATION

SUBTRACT Instruction (SUB/RSB/SBCS)

1 Subtract two registers

SUBS Rd, Rn, Rm ; $Rd = Rn - Rm$, APSR update.

;Rd, Rn, Rm are low registers

2 Subtract a register with an immediate constant

SUBS Rd, Rn, #mmed3 ; $Rd = Rn - \text{ZeroExtend}(\#mmed3)$,
;APSR update

SUBS Rd, #mmed8 ; $Rd = Rd - \text{ZeroExtend}(\#mmed8)$,
;APSR update
;Rd, Rn, Rm are low registers

3 Subtract with carry (borrow)

SBCS Rd, Rd, Rm ; $Rd = Rd - Rm - \text{Borrow}$, APSR update

4 Reverse Subtract (negative)

RSBS Rd, Rm, #0 ; $Rd = 0 - Rm$, APSR update



ARITHMETIC OPERATION

Compare Instructions (CMP/CMN)

There are also a few compare instructions that compare (using subtract) values and update flags (APSR), but the result of the compare is not stored.

1. Compare Register value

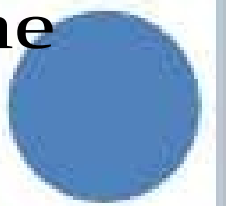
CMP Rn, Rm ; Calculate $Rn - Rm$,
; APSR update but subtract result is not stored.

2. Compare Register value with immediate data

CMP Rn, #mmed8
; Calculate $Rd - \text{ZeroExtended}(\#mmed8)$, APSR update
; but subtract result is not stored. Rn is a low registers.

3. Compare negative

CMN Rn, Rm
; Calculate $Rn - \text{NEG}(Rm)$,
; APSR update but subtract result is not stored. Effectively the
; operation is an ADD.



ARITHMETIC OPERATION

MUL Instruction

MUL S <Rd>, <Rm>, <Rd>

;Rd=Rd * Rm,

;A PSR.N and APSR.Z update

;Rd and Rm are low registers



LOGICAL OPERATIONS

Instruction	Function	Syntax	Note
AND	Logical AND	ANDS Rd, Rd, Rm	$Rd = A \text{ AND}(Rd, Rm)$,
ORR	Logical OR	ORRS Rd, Rd, Rm	$Rd = OR(Rd, Rm)$,
EOR	Logical Exclusive OR	EORS Rd, Rd, Rm	$Rd = X \text{ OR}(Rd, Rm)$,
BIC	Logical Bitwise Clear	BICS Rd, Rd, Rm	$Rd = A \text{ AND}(Rd, \text{NOT}(Rm))$
MVN	Logical Bitwise NOT	MVNS Rd, Rm	$Rd = \text{NOT}(Rm)$
TST	Test (bitwise AND)	TST Rd, Rm	Calculate $A \text{ AND}(Rn, Rm)$ But the AND result is not stored

In all logical operations

Rd and Rm are low registers

APSR.N and APSR.Z update



SHIFT AND ROTATE OPERATIONS

The Cortex-M0 and Cortex-M0+ Processors supports both arithmetic shift operations (data is a signed integer value where MSB needs to be reserved) as well as logical shift.

Instruction	Function	Syntax	Note
ASR	Arithmetic Shift Right	ASRS Rd, Rd, Rm	$Rd = Rd \gg Rm$
	ASR with immediate Data	ASRS Rd, Rm, #immed5	$Rd = Rm \gg immed5$
LSL	Logical Shift Left	LSLS Rd, Rd, Rm	$Rd = Rd \ll Rm$
	LSL with immediate Data	LSLS Rd, Rm, #immed5	$Rd = Rm \ll immed5$
LSR	Logical Shift Right	LSRS Rd, Rd, Rm	$Rd = Rd \gg Rm$
	LSR with immediate Data	LSRS Rd, Rm, #immed5	$Rd = Rm \gg immed5$



REVERSE ORDERING OPERATIONS

Reverse instructions are usually used for converting data between little endian and big endian systems. These instruction includes

1. REV (Byte Reverse in Word, Figure 1),
2. REV16 (Byte Reverse Packed Half Word, Figure 2),
3. REVSH (Byte Reverse Signed Half Word, Figure 3)

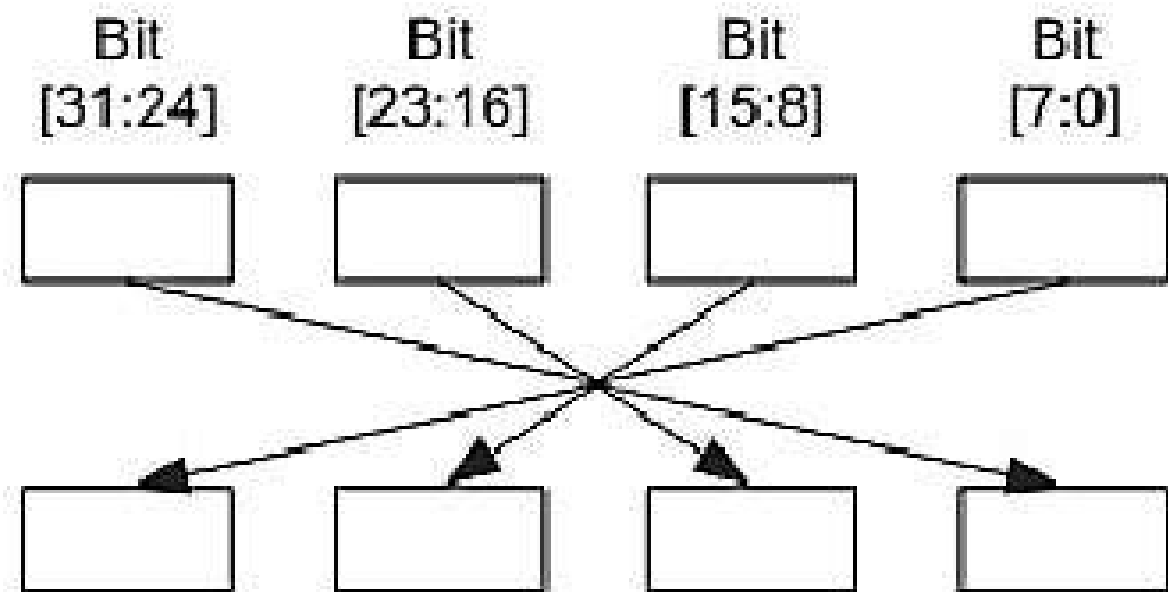


Figure 1: REV operation

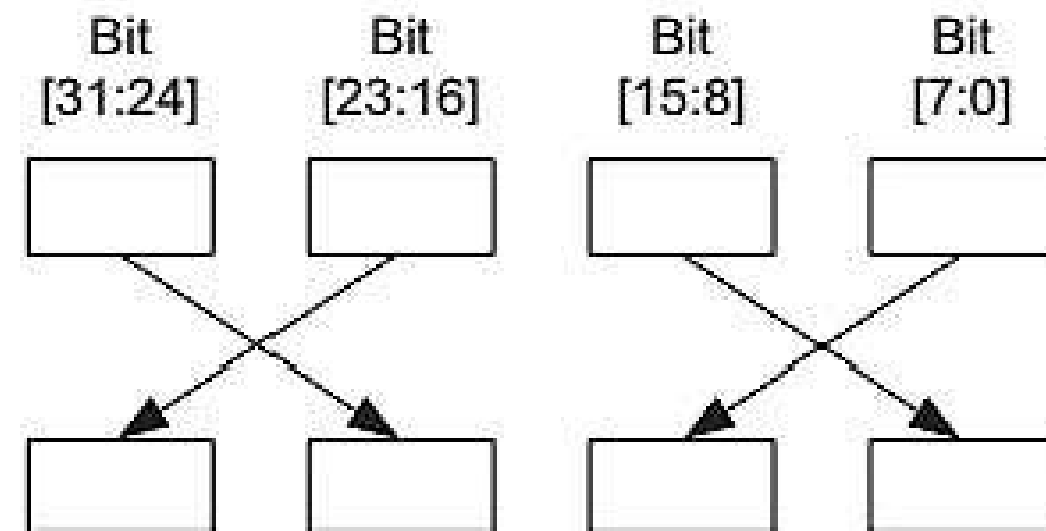


Figure 2: REV16 operation

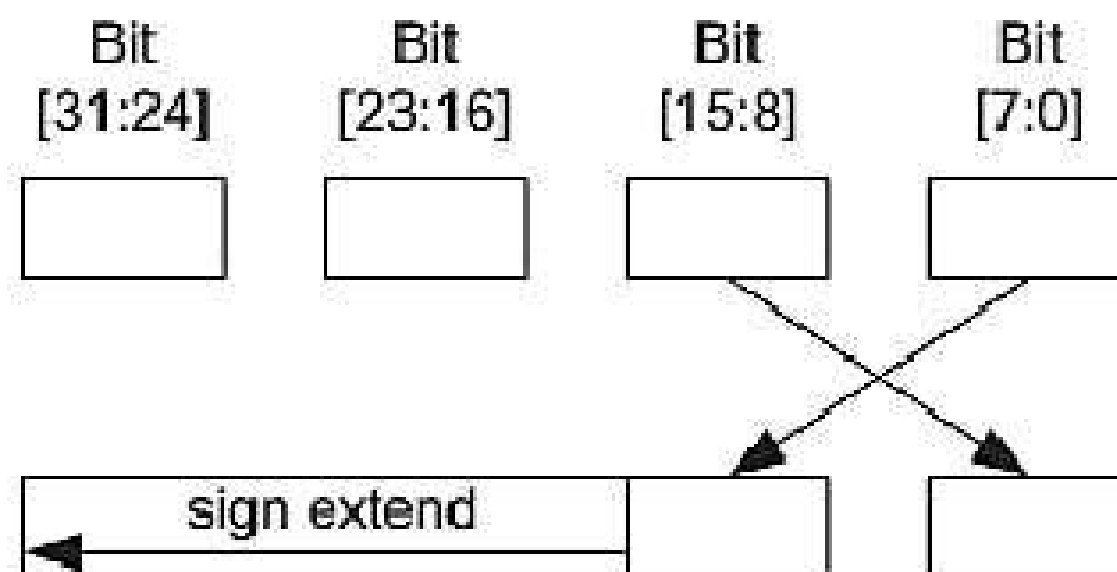
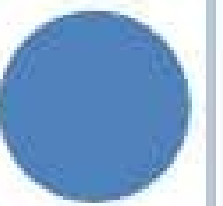


Figure 3: REVSH operation

REVERSE ORDERING OPERATIONS

Instruction	Function	Syntax	Note
REV	Byte Order Reverse	REV Rd, Rm	$Rd = \{Rm[7:0], Rm[15:8], Rm[23:16], Rm[31:24]\}$
REV16	Byte Order Reverse within half word	REV16 Rd, Rm	$Rd = \{Rm[23:16], Rm[31:24], Rm[7:0], Rm[15:8]\}$
REVSH	Byte order reverse within lower half word, then sign extend result	REVSH Rd, Rm	$Rd = \text{SignExtend}(\{Rm[7:0], Rm[15:8]\})$

In all reverse ordering operations Rd and Rm are low registers.



EXTENDING ORDERING OPERATIONS

The SXTB, SXTH, UXT, and UXTH instructions are used for extending a byte or half word data into a word. They are usually used for data type conversions.

Instruction	Function	Syntax	Note
SXTB	SignExtend lowest byte in a word of data	SXTB Rd, Rm	Rd = SignExtend(Rm[7:0])
SXTH	SignExtend lower half word in a word of data	SXTH Rd, Rm	Rd = SignExtend(Rm[15:0])
UXTB	Extend lowest byte in a word of data	UXTB Rd, Rm	Rd = ZeroExtend(Rm[7:0])
UXTH	Extend lower half word in a word of data	UXTH Rd, Rm	Rd = ZeroExtend(Rm[15:0])

In all Extending ordering operations Rd and Rm are low registers.

EXTENDING ORDERING OPERATIONS

With SXTB or SXTH, the data is extended using bit[7] or bit[15] of the input data.

While for UXTB and UXTH, the data is extended using zeros.

For example, if R0 is 0x55AA8765, and the result of these extended instructions are

SXTB R1, R0 ; R1 = 0x00000065 (bit7 is zero)

SXTH R1, R0 ; R1 = 0xFFFF8765 (bit15 is one)

UXTB R1, R0 ; R1 = 0x00000065

UXTH R1, R0 ; R1 = 0x00008765



PROGRAM FLOW CONTROL OPERATIONS

There are five branch instructions in the Cortex-M0 and Cortex-M0+ processors.

They are essential for program flow control like looping and conditional execution, and allow program code to be partitioned into functions and subroutines.

Instruction: **B (Branch)**

Function: Branch to an address (unconditional)

Syntax: **B** <label>

Note: Branch range is ± 2046 bytes of current program counter

For example: **B** Loop ;Branch to location Loop

Instruction: **B<cond> (Conditional Branch)**

Function: Depending of A PSR, branch to an address

Syntax: **B<cond>** <label>

Note: Branch range is ± 254 bytes of current program counter.

For example:

CMP R0, #0x1 ; Compare R0 with 0x1

BEQ process1 ; Branch to process1 if R0 equal 1



PROGRAM FLOW CONTROL OPERATIONS

The <cond> is one of the 14 possible condition suffixes (Table 1).

Suffix	Branch condition	Flags (APSR)
EQ	Equal	Z flag is set
NE	Not equal	Z flag is cleared
CS/HS	Carry set/unsigned higher or same	C flag is set
CC/LO	Carry clear/unsigned lower	C flag is cleared
MI	Minus/negative	N flag is set (minus)
PL	Plus/positive or zero	N flag is cleared
VS	Overflow	V flag is set
VC	No overflow	V flag is cleared
HI	Unsigned higher	C flag is set and Z is cleared
LS	Unsigned lower or same	C flag is cleared or Z is set
GE	Signed greater than or equal	N flag is set and V flag is set, or N flag is cleared and V flag is cleared (N == V)
LT	Signed less than	N flag is set and V flag is cleared, or N flag is cleared and V flag is set (N != V)
GT	Signed greater then	Z flag is cleared, and either both N flag and V flag are set, or both N flag and V flag are cleared (Z == 0 and N == V)
LE	Signed less than or equal	Z flag is set, or either N flag set with V flag cleared, or N flag cleared and V flag set (Z == 1 or N != V)

Table 1: Condition suffixes for conditional branches

PROGRAM FLOW CONTROL OPERATIONS

Instruction: BL (Branch and Link)

Function: Branch to an address and store return address to Link Register. Usually use for function calls, and can be used for long range branch that is beyond the branch range of branch instruction (B <label>).

Syntax: BL <label>

Note: Branch range is 16 MB of current program counter.

For example:

BL functionA ; call a function called functionA

Instruction: BX (Branch and Exchange)

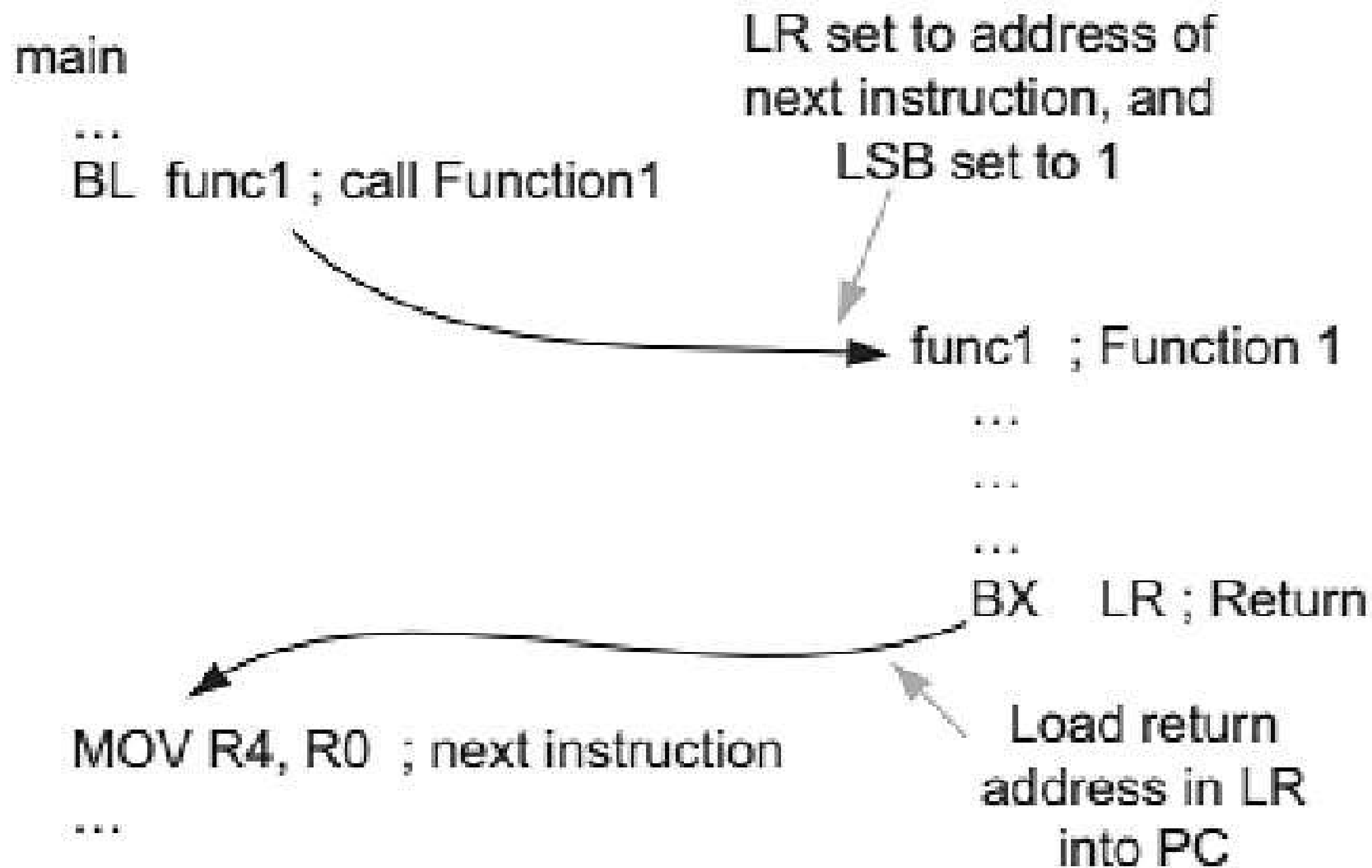
Function: Branch to an address specified by a register, and change processor state depending on bit[0] of the register

Syntax: BX <Rm>

Note: Since the Cortex-M processors only supports Thumb code, bit[0] of the register content (Rm) must be set to 1, otherwise it means that it is trying to switch to ARM state and this will generate a fault exception

PROGRAM FLOW CONTROL

Function call and return using BL and BX instructions.



DIRECTIVES

They assist and control assembly process.

They are also called pseudo-ops.

They are not the part of instruction set.

They changed the way the code is assembled.

Commonly used directives are:

Directive	Function
THUMB	Placed at the top of the file to specify that code is generated with thumb instructions.
CODE	Denotes the section for machine instructions (ROM).
DATA	Denotes the section for global variables (RAM).
AREA	Instruct the assembler to assemble a new code or data section.

DIRECTIVES

Directive	Function
END	Placed at the end of the each file.
DCB	Places byte (8-bit) sized constant in memory.
DCW	Place a half word (16 bits) sized constant into memory.
DCD	places a word (32 bits) sized constant into memory.
EQU	to give the symbolic name to numeric constant.



TEMPLATE OF ARM ASSEMBLY LANGUAGE PROGRAM

THUMB ; marks the THUMB mode of operation

AREA mydata, DATA

<Define the global variables>

AREA mycode, CODE ; starting point of the code execution

EXPORT __main ; declaration of identifier main

__main ; address of the main function

<write the User Code>

END ; End of the program

ARM ASSEMBLY LANGUAGE PROGRAM EXAMPLE

AREA mydata, DATA

Sum EQU 0 ; declaration of Global variables Sum and Count

Count EQU 5

area mycode, CODE

export __main

__main

MOV S R0, #Sum ; R0=Sum = 0

MOV S R1, #Count ; R1=Count = 5

Loop

ADDS R0, R0, R1 ; R0=R0+R1 , means (Sum = Sum + Count)

subs R1, R1, #1 ; Count = Count - 1

BNE Loop ; if R1>0 then branch to loop

end

Thanks

